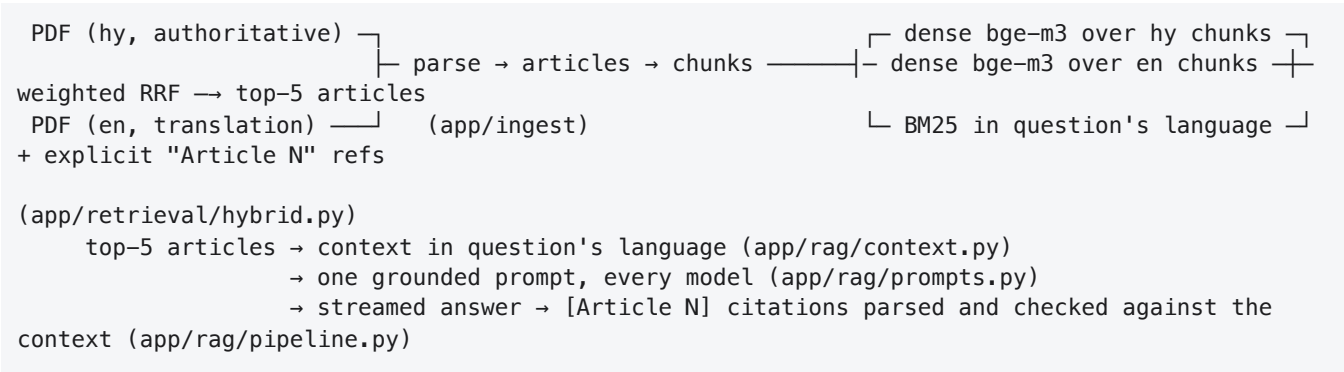


RAG Pipeline — Design and Rationale

System: Q&A assistant for the Law of the Republic of Armenia "On Electronic Communications" (HO-176-N)
Users: regulatory affairs and legal teams; questions in Armenian or English **Goal:** accurate answers grounded in the law, with citations to specific articles



Everything runs locally except the LLM call. Retrieval takes about 0.15 s on a laptop CPU.

1. Document ingestion — app/ingest/parse.py

Sources. Both versions from arlis.am are used:

	Armenian PDF	English PDF
Role	Authoritative text. The task links the /hy/ page, and the system must handle Armenian natively.	Parallel translation for English questions and cross-lingual retrieval
Edition	Original 2005 text (arlis.am shows the status "in force 03.09.2005–07.04.2007")	Consolidated translation that includes amendments of 2007 (HO-90-N, HO-160-N) and 2009 (HO-207-N)
Articles	1–67	1–67 plus Article 17.1 (radio-frequency fees, added in 2009)

Because the articles align one-to-one by number, a hit in either language points to the same article. The two versions are **not** identical in content. The English text contains amendments, including Article 17.1, that the Armenian original lacks. This is kept visible, not hidden (see Limitations).

Extraction. I used PyMuPDF instead of pypdf, because pypdf split words inside tokens in the English PDF ("commu nications", "el ectronic"). That damages both BM25 and the text the LLM quotes.

Structure recovery. Laws have a strict hierarchy: chapter → (section) → article → numbered part → item. The parser: - drops the arlis.am metadata page and the amendment-history table (Armenian), page numbers, and the signature block; - recognises Գ Լ ՈՒ Խ Ն / Շ Ի Մ Կ Ե Ր Ն and Բ Ա Ժ Ի Ն / Տ Ե Կ Ե Լ Ո Ն Ն headers, and keeps their titles out of the article bodies; - detects article headers (Հոդված N. / Article N.) but **rejects in-text references**. "...Article 5(2)(z) and ↵ Article 10." looks like a header after a line break, so a header is accepted only if the previous line ended a sentence; - joins titles that wrap onto a second line and re-flows wrapped lines into paragraphs, where a paragraph starts at a list marker (1., (2), 2), ա)); - splits each article into its top-level numbered parts.

Validation. Ingestion fails loudly unless articles 1–67 are all present, in order, with no duplicates or empty bodies, in both languages. The parsed articles are committed (data/processed/), so the app runs without the PDFs.

2. Chunking — app/ingest/chunk.py

Decision	Why
A chunk never crosses an article boundary	Citations are at article level. A chunk mixing two articles would make the citation ambiguous and pull in off-topic text.
Whole numbered parts are packed greedily up to 1,200 characters	Most articles fit in one chunk. Long ones split at their own part boundaries, which are the law's own semantic units.
Over-long parts are split at clause ends (; . : :)	Needed mainly for Article 2 (definitions), which is one long list.
Each chunk's embedding text starts with Article N. Title (Chapter ...)	A chunk from the middle of an article still carries its topic. Legal article titles are dense summaries.
No overlap	The part structure already gives clean boundaries, and overlap would duplicate text in the context.

Result: 148 Armenian and 149 English chunks (average ~800 characters, maximum 1,204). About 800 characters is 150–250 English tokens, which suits bge-m3 and keeps each chunk on a single legal point.

3. Retrieval — app/retrieval/

Embedding model: BAAI/bge-m3, run locally with the revision pinned: - It is multilingual, handles Armenian, and is **cross-lingual**: an English question lands near the Armenian text and the other way round. In a quick check, the same question in hy and en scored a cosine similarity of 0.65, against about 0.3 for unrelated text. - Running it locally avoids spending free-tier API quota and avoids rate limits, and the results are deterministic. - It supports an 8k-token context, so no chunk is truncated.

Vector store: a numpy matrix. 297 × 1024 floats with exact cosine search takes under a millisecond. A vector database would add infrastructure with no gain at this scale. Vectors are cached on disk and keyed by the model revision and a hash of the chunk texts.

Hybrid retrieval with weighted Reciprocal Rank Fusion. There are three ranked lists: 1. dense search over the Armenian chunks; 2. dense search over the English chunks; 3. BM25 over chunks in the question's language. Tokens are prefix-stemmed to 6 characters, because Armenian is highly inflected (լիցենզիա, լիցենզիայի, լիցենզիաների → լիցենզ).

Each list is first collapsed to an **article ranking**, so an article counts once, at its best chunk. This stops long articles from winning because they have many chunks. The rankings are then fused with RRF ($k = 60$), which only uses ranks, so scores that aren't comparable (BM25 vs. cosine) never need calibrating. Two further rules: - **Explicit references** ("Article 45", "Հոդված 45", "հոդ. 45") force that article into the result. Users of a legal tool often ask about an article by number. - **Single-language articles.** Article 17.1 exists only in the English text, so it can appear in only one of the two dense lists and would always be outranked. It gets the same contribution in the missing list. Without this fix, the 17.1 question retrieved articles 10, 17 and 13 instead.

Evidence for the fusion weights (uv run python -m app.eval.retrieval_eval). I used two question sets: the 17 answerable benchmark questions, which are worded close to the law, and a separate **paraphrase dev set** of 18 colloquial questions (eval/retrieval_dev.json), e.g. "Can my phone company cut me off if I'm late paying my bill?".

Article recall@5	Benchmark questions	Paraphrase dev set
Dense only	0.892	1.000

Article recall@5	Benchmark questions	Paraphrase dev set
BM25 only	1.000	0.611
Hybrid, equal weights	0.922	0.889
Hybrid, BM25 weight 0.25 (chosen)	0.941	0.944

BM25 is excellent when the question uses the law's own words and weak on paraphrases. Dense retrieval is the reverse. Real users will do both. Dense is therefore the primary signal, and BM25 adds a small lexical boost for exact legal terms, numbers and Armenian word forms. I chose the weight after seeing both sets, and the sets are small (a 0.06 difference is one question), so this is a reasoned default, not a tuned optimum. **Top-k = 5 articles**: recall rises from k = 3 to k = 5, and more articles would mainly add prompt tokens.

4. Context assembly — `app/rag/context.py`

- **The unit is the article, in retrieval order**, wrapped as `<article number="45" title="...">`, so the model cites exactly what it sees.
- **Language: the text is in the question's language**. Armenian questions get the authoritative Armenian original. English questions get the English translation. If an article exists in only one version (17.1), that version is used. I didn't send both versions of every article because it roughly doubles prompt tokens (Armenian script costs about 2–3x more tokens per character) for little gain, since every benchmarked model reads both languages.
- **Short articles ($\leq 3,500$ characters) are included whole**. Exceptions and definitions are often in a different part than the one that matched. Article 45(2) (notice period) only makes sense next to 45(1) (grounds). Longer articles contribute only their matched chunks (at most 3), mapped to the answer language by part number and marked [excerpt].
- **Budget: 14,000 characters**. Every model gets the same bounded prompt, and it stays inside free-tier tokens-per-minute limits. Groq's gpt-oss-120b free tier allows 8K tokens per minute.

5. Answer generation — `app/rag/prompts.py`, `app/rag/pipeline.py`, `app/llm/providers.py`

One prompt for every model (temperature 0), so the benchmark compares models, not prompts. Rules: 1. Use only the excerpts in `<context>`. 2. End every factual sentence with a citation in a fixed, parseable format: [Article 45], [Article 45(2)], or [Հոդված 45] in Armenian. 3. Answer in the question's language, even if the excerpt is in the other language. 4. If the context does not answer the question, reply with one **fixed refusal sentence** (one per language). If only part is covered, answer that part and say what is missing. 5. Keep numbers, deadlines and legal terms exactly as written in the law.

A fixed refusal string means out-of-scope handling can be scored **deterministically**, with no judge needed.

Post-processing. Citations are parsed from the answer. The parser is tolerant, so [Article 12, 13] still works. Each citation is checked against the articles that were actually in the context. A citation of an article that was not retrieved is a hallucination signal: the UI shows it in red and the benchmark counts it.

Provider layer. Gemini, Groq and OpenRouter all expose OpenAI-compatible chat endpoints, so a single streaming client handles all three. The request shape, timing code and error handling are identical for each. Per call it records: - **TTFT**, measured to the first content token; - total time; - **token usage** as reported by the API. If a provider omits it, a character-based estimate is used and flagged; - the **failure category** (rate_limit, timeout, api_error, empty_output, missing_key).

Rate limits and timeouts are retried with backoff only *before* the first token. A client-side **pac**er keeps each model inside its free-tier RPM and TPM, so the benchmark measures the models, not a burst of 429 errors.

Streaming to the browser uses Server-Sent Events: first the retrieved articles, then tokens, then a final event with citations, latency, tokens and cost at paid rates.

Where to look in the code

Mechanism	File
Header detection, in-text reference rejection, validation	app/ingest/parse.py (parse, validate)
Part-aware chunking	app/ingest/chunk.py (chunk_article)
Dense index and cache	app/retrieval/embeddings.py (DenseIndex)
Language detection, Armenian stemming, weighted RRF, forced refs, single-language fix	app/retrieval/hybrid.py (HybridRetriever.retrieve)
Language-aware context and budget	app/rag/context.py (build_context)
Prompt, citation parser, refusal detection	app/rag/prompts.py
Streaming, TTFT, usage, retries, pacing	app/llm/providers.py (stream_chat, Pacer)
Retrieve once, generate per model	app/rag/pipeline.py (prepare, generate)

Known limitations of the pipeline

- **Law version.** The Armenian source is the original 2005 edition, and the English one is consolidated to about 2009. The law has since been amended many times (the arlis.am history lists changes up to 2025). Answers reflect these texts, not the law currently in force. Production would ingest the current consolidated version and store the edition date with each chunk.
- **Asymmetric content.** Armenian questions about Article 17.1 depend on cross-lingual dense retrieval, because the Armenian original has no 17.1.
- **Small evaluation sets** (17 + 18 retrieval queries). Differences of one or two questions are within noise.
- **No reranker.** A cross-encoder (e.g. bge-reranker-v2-m3) would probably fix the remaining misses, where the right article is at rank 6–7, but it adds about 1 s of CPU latency per question.
- **Chunk-to-language mapping** for long articles uses part numbers. It is exact when the parts align (they do in this law) and falls back to chunk position otherwise.